PRACTICA 3 + EXTRA

December 31, 2023

Vamos a comentar las partes más relevantes del código. Comentar que se juntarán en el mismo notebook tanto el ejercicio principal como el extra. Al tratarse de un notebook me veo obligado a proporcionar el código completo del primer script. Al final mostraré las modificaciones principales a este primero para la resolución del ejercicio extra.

Se definen primeramente las librerias que nos serán necesarias junto a las variables que aparecerán en las diferentes operaciones que se realizarán en los bucles y funciones que más abajo se mostrarán. Los valores de estas son los referenciados en el enunciado de la práctica.

[9]:
```
import numpy as np # Para hacer cálculos numéricos
import matplotlib.pyplot as plt # Para hacer gráficos
from mpl_toolkits.mplot3d import Axes3D # Para hacer gráficos en 3D
import math # Para usar funciones matemáticas

# Definir las constantes y los parámetros del problema
sigma = 2.3151 # Parámetro de longitud del potencial de Lennard-Jones en Å
epsilon = 0.167 # Parámetro de energía del potencial de Lennard-Jones en eV
a = 3.603 # Parámetro de red de la red f.c.c. en Å
rc = 3 * sigma # Radio de corte del potencial en Å
n = 12 # Exponente repulsivo del potencial
m = 6 # Exponente atractivo del potencial
```

Se definen varias funciones, las cuales realizarán las tareas necesarias para la modelización del problema.

La función `generar_red_fcc` se utiliza para generar una red de celdas unitarias en una estructura FCC.

La función toma cuatro argumentos: `nx`, `ny`, y `nz` son el número de celdas unitarias en las direcciones x, y, y z, respectivamente, y `a` es el parámetro de red, que es la longitud de los lados de la celda unitaria.

Se calcula el número total de átomos en la red (cada celda unitaria en una red FCC contiene 4 átomos) y se crea una matriz de ceros para almacenar las posiciones de los átomos.

Se recorren todas las celdas unitarias en la red. Para cada celda unitaria, se calculan las coordenadas de la esquina inferior izquierda de la celda y se añaden las posiciones de los cuatro átomos en la celda a la matriz de posiciones.

Después de colocar cada átomo, se calcula el porcentaje de átomos que aún no se han colocado y se imprime este valor.

Finalmente, la función devuelve la matriz de posiciones de los átomos.

En resumen, `generar_red_fcc` genera una red FCC de átomos y devuelve sus posiciones.

[10]:
```
def generar_red_fcc(nx, ny, nz, a):
  # Crear una matriz vacía para almacenar las posiciones de los átomos
  N = 4 * nx * ny * nz # Número total de átomos
  pos = np.zeros((N, 3)) # Matriz de N filas y 3 columnas
  # Recorrer las celdas unidad en cada dirección
  i = 0 # Índice para contar los átomos
  print("Generando red f.c.c....")
  for ix in range(nx):
    for iy in range(ny):
      for iz in range(nz):
        # Calcular las coordenadas de la esquina inferior izquierda de la celda␣
↪unidad
        x0 = ix * a
        y0 = iy * a
        z0 = iz * a
        # Añadir las posiciones de los cuatro átomos de la celda unidad
        pos[i] = [x0, y0, z0] # Átomo en la esquina
        i += 1
        pos[i] = [x0 + 0.5 * a, y0 + 0.5 * a, z0] # Átomo en el centro de la␣
↪cara xy
        i += 1
        pos[i] = [x0, y0 + 0.5 * a, z0 + 0.5 * a] # Átomo en el centro de la␣
↪cara yz
        i += 1
        pos[i] = [x0 + 0.5 * a, y0, z0 + 0.5 * a] # Átomo en el centro de la␣
↪cara xz
        i += 1
        # Calcular el porcentaje de átomos que quedan por colocar
        porcentaje = (N - i) / N * 100
        print(f"Porcentaje de átomos restantes: {porcentaje:.2f}%")
  # Devolver la matriz de posiciones
  return pos
```

La función `distancia` calcula la distancia entre dos átomos en una red de celdas unitarias. Esta función es esencial para calcular las interacciones entre los átomos en la red.

Primero, la función extrae las posiciones de los átomos `i` y `j` de la matriz `pos`. Luego, calcula la distancia euclídea entre los dos átomos.

Si se deben aplicar condiciones de contorno periódicas, la función corrige la distancia entre los átomos para tener en cuenta la imagen más cercana. Esto se hace calculando las longitudes de la 'caja' en las tres direcciones y luego ajustando las componentes de la distancia si son mayores que la mitad de la longitud de la caja correspondiente. Luego, se recalcula la distancia con los valores corregidos.

Finalmente, la función devuelve la distancia entre los dos átomos.

[11]:
```python
def distancia(i, j, pos, nx, ny, nz, a, periodicas):
  # Extraer las posiciones de los átomos i y j
  xi, yi, zi = pos[i]
  xj, yj, zj = pos[j]
  # Calcular la distancia euclídea entre los átomos
  dx = xj - xi
  dy = yj - yi
  dz = zj - zi
  r = math.sqrt(dx**2 + dy**2 + dz**2)
  # Si se aplican condiciones periódicas, corregir la distancia según la imagen␣
↪más cercana
  if periodicas:
    Lx = nx * a # Longitud de la caja en la dirección x
    Ly = ny * a # Longitud de la caja en la dirección y
    Lz = nz * a # Longitud de la caja en la dirección z
    # Si la distancia es mayor que la mitad de la longitud de la caja, restar la␣
↪longitud de la caja
    if dx > 0.5 * Lx:
      dx -= Lx
    elif dx < -0.5 * Lx:
      dx += Lx
    if dy > 0.5 * Ly:
      dy -= Ly
    elif dy < -0.5 * Ly:
      dy += Ly
    if dz > 0.5 * Lz:
      dz -= Lz
    elif dz < -0.5 * Lz:
      dz += Lz
    # Recalcular la distancia corregida
    r = math.sqrt(dx**2 + dy**2 + dz**2)
  # Devolver la distancia
  return r
```

Las funciones `energia_interaccion` y `energia_total` se utilizan para calcular la energía de interacción entre dos átomos y la energía total del sistema, respectivamente.

La función `energia_interaccion` toma como argumentos los índices de dos átomos, sus posiciones en la red, el número de celdas unitarias en cada dirección, el parámetro de red, un booleano que indica si se deben aplicar condiciones de contorno periódicas, los parámetros del potencial de Lennard-Jones (sigma y epsilon), los exponentes del potencial (n y m), y el radio de corte. Esta función calcula la distancia entre los dos átomos y, si esta distancia es menor que el radio de corte, calcula la energía de interacción utilizando la fórmula del potencial de Lennard-Jones. Si la distancia es mayor o igual que el radio de corte, la energía de interacción es cero.

La función `energia_total` toma como argumentos las posiciones de todos los átomos en la red, el número de celdas unitarias en cada dirección, el parámetro de red, un booleano que indica si se deben aplicar condiciones de contorno periódicas, los parámetros del potencial de Lennard-Jones (sigma y epsilon), los exponentes del potencial (n y m), y el radio de corte. Esta función recorre

todos los pares de átomos en la red y, para cada par, calcula la energía de interacción utilizando la función `energia_interaccion` y la suma a la energía total del sistema. Finalmente, devuelve la energía total.

[12]:
```
def energia_interaccion(i, j, pos, nx, ny, nz, a, periodicas, sigma, epsilon, n,␣
↪m, rc):
  # Calcular la distancia entre los átomos i y j
  r = distancia(i, j, pos, nx, ny, nz, a, periodicas)
  # Si la distancia es menor que el radio de corte, calcular la energía de␣
↪interacción según el potencial de Lennard-Jones
  if r < rc:
    u = 4 * epsilon * ((sigma / r)**n - (sigma / r)**m) # Fórmula del potencial
  # Si la distancia es mayor o igual que el radio de corte, la energía de␣
↪interacción es cero
  else:
    u = 0
  # Devolver la energía de interacción
  return u

# Definir una función que calcule la energía total del sistema como la suma de␣
↪las energías de interacción de todos los pares de átomos, teniendo en cuenta␣
↪las condiciones periódicas o de superficie libre
def energia_total(pos, nx, ny, nz, a, periodicas, sigma, epsilon, n, m, rc):
  # Crear una variable para almacenar la energía total
  U = 0
  # Recorrer todos los pares de átomos
  N = len(pos) # Número total de átomos
  for i in range(N):
    for j in range(i + 1, N): # Evitar repetir pares y contar el mismo átomo
      # Calcular la energía de interacción entre los átomos i y j y sumarla a la␣
↪energía total
      U += energia_interaccion(i, j, pos, nx, ny, nz, a, periodicas, sigma,␣
↪epsilon, n, m, rc)
  # Devolver la energía total
  return U
```

Ya por último, se define una función para graficar la red tridimensional. A destacar que a cada átomo se le asociará un color en función de su 'profundidad' en la red. Vamos a suponer sicerta orientación en el cristal, y con cierta analogía a la vida real pues disponiendo el cristal una parte quedará orientada hacia 'arriba' y otra hacia 'abajo'.

[13]:
```
def graficar_red(pos):
  # Crear una figura y un eje 3D
  fig = plt.figure()
  ax = fig.add_subplot(111, projection='3d')
  # Extraer las coordenadas x, y, z de la matriz de posiciones
  x = pos[:, 0]
```

```
  y = pos[:, 1]
  z = pos[:, 2]
  # Calcular el color de cada átomo según su profundidad
  c = z / max(z)
  # Representar los átomos como puntos de colores
  scatter = ax.scatter(x, y, z, c=c, cmap='viridis')

  cbar = plt.colorbar(scatter)
  cbar.set_label('Profundidad')
  # Ajustar los límites de los ejes
  ax.set_xlim(min(x), max(x))
  ax.set_ylim(min(y), max(y))
  ax.set_zlim(min(z), max(z))
  # Etiquetar los ejes
  ax.set_xlabel('x (Å)')
  ax.set_ylabel('y (Å)')
  ax.set_zlabel('z (Å)')
  # Mostrar el gráfico
  plt.show()
```

Por último, el código interactúa con el usuario solicitando varios parámetros necesarios para la simulación de dinámica molecular.

Primero, se solicita al usuario que introduzca el número de celdas unitarias en las direcciones x, y y z. Estos valores son convertidos a enteros y almacenados en las variables `nx`, `ny` y `nz`, respectivamente.

Luego, se pide al usuario que introduzca el tipo de condiciones de contorno, que pueden ser "periódicas" o "superficies libres". La respuesta del usuario se convierte a minúsculas y se compara con las cadenas "periódicas" y "periodicas" para determinar si se deben aplicar condiciones de contorno periódicas. El resultado se almacena en la variable booleana `periodicas`.

Si el usuario no introduce "periódicas" o "superficies libres", se muestra un mensaje de error y se solicita al usuario que introduzca de nuevo el tipo de condiciones de contorno. Si el usuario vuelve a introducir un valor no válido, se asumen condiciones de superficies libres.

Finalmente, se informa al usuario del tipo de condiciones de contorno que se han aplicado.

[14]:
```
nx = int(input("Introduzca el número de celdas unidad en la dirección x: "))
ny = int(input("Introduzca el número de celdas unidad en la dirección y: "))
nz = int(input("Introduzca el número de celdas unidad en la dirección z: "))
condiciones = input("Introduzca el tipo de condiciones de contorno (periódicas o␣
 ↪superficies libres): ")
# Convertir el tipo de condiciones a un valor booleano
periodicas = condiciones.lower() == "periódicas" or condiciones.lower() ==␣
 ↪"periodicas"
# Verificar si el usuario ha introducido periódicas o superficies libres
if not periodicas:
  if condiciones.lower() != "superficies libres":
```

```
    print("Por favor, introduzca 'periódicas' o 'superficies libres' como tipo␣
↪de condiciones de contorno.")
    print('En caso de que no se introduzca un tipo de condiciones de contorno␣
↪válido, se asumirán condiciones de superficies libres.')
    condiciones = input("Introduzca el tipo de condiciones de contorno␣
↪(periódicas o superficies libres): ")
    periodicas = condiciones.lower() == "periódicas" or condiciones.lower() ==␣
↪"periodicas"

# Aplicar las funciones definidas anteriormente para generar la red, calcular la␣
↪energía total y mostrar el gráfico

pos = generar_red_fcc(nx, ny, nz, a)
U = energia_total(pos, nx, ny, nz, a, periodicas, sigma, epsilon, n, m, rc)
graficar_red(pos)
# Mostrar la energía total por átomo
print(f"La energía total del sistema por átomo es {U / len(pos)} eV.")
if not periodicas:
  condiciones = "superficies libres"
print('Se eligió como condiciones de contorno:', condiciones)
```

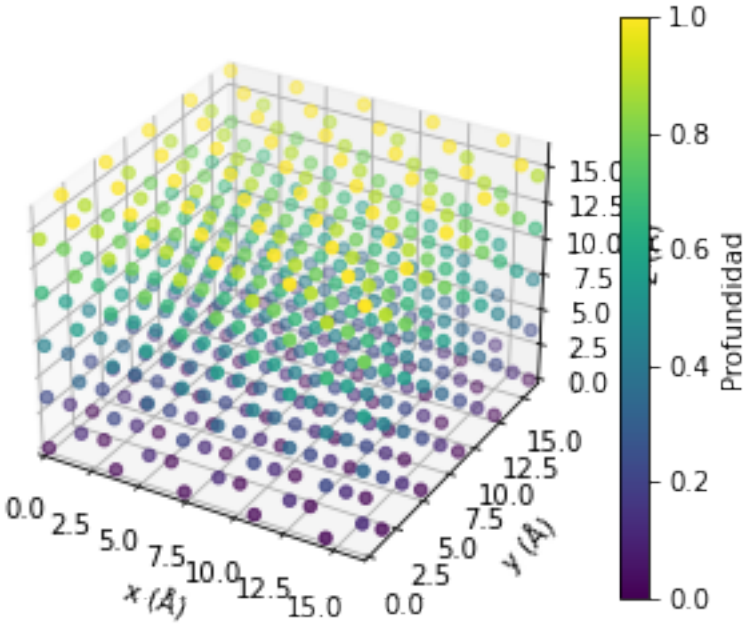
```
Por favor, introduzca 'periódicas' o 'superficies libres' como tipo de
condiciones de contorno.
En caso de que no se introduzca un tipo de condiciones de contorno válido, se
asumirán condiciones de superficies libres.
Generando red f.c.c...
Porcentaje de átomos restantes: 99.20%
Porcentaje de átomos restantes: 98.40%
Porcentaje de átomos restantes: 97.60%
Porcentaje de átomos restantes: 96.80%
Porcentaje de átomos restantes: 96.00%
Porcentaje de átomos restantes: 95.20%
Porcentaje de átomos restantes: 94.40%
Porcentaje de átomos restantes: 93.60%
Porcentaje de átomos restantes: 92.80%
Porcentaje de átomos restantes: 92.00%
Porcentaje de átomos restantes: 91.20%
Porcentaje de átomos restantes: 90.40%
Porcentaje de átomos restantes: 89.60%
Porcentaje de átomos restantes: 88.80%
Porcentaje de átomos restantes: 88.00%
Porcentaje de átomos restantes: 87.20%
Porcentaje de átomos restantes: 86.40%
Porcentaje de átomos restantes: 85.60%
Porcentaje de átomos restantes: 84.80%
Porcentaje de átomos restantes: 84.00%
Porcentaje de átomos restantes: 83.20%
```

```
Porcentaje de átomos restantes: 82.40%
Porcentaje de átomos restantes: 81.60%
Porcentaje de átomos restantes: 80.80%
Porcentaje de átomos restantes: 80.00%
Porcentaje de átomos restantes: 79.20%
Porcentaje de átomos restantes: 78.40%
Porcentaje de átomos restantes: 77.60%
Porcentaje de átomos restantes: 76.80%
Porcentaje de átomos restantes: 76.00%
Porcentaje de átomos restantes: 75.20%
Porcentaje de átomos restantes: 74.40%
Porcentaje de átomos restantes: 73.60%
Porcentaje de átomos restantes: 72.80%
Porcentaje de átomos restantes: 72.00%
Porcentaje de átomos restantes: 71.20%
Porcentaje de átomos restantes: 70.40%
Porcentaje de átomos restantes: 69.60%
Porcentaje de átomos restantes: 68.80%
Porcentaje de átomos restantes: 68.00%
Porcentaje de átomos restantes: 67.20%
Porcentaje de átomos restantes: 66.40%
Porcentaje de átomos restantes: 65.60%
Porcentaje de átomos restantes: 64.80%
Porcentaje de átomos restantes: 64.00%
Porcentaje de átomos restantes: 63.20%
Porcentaje de átomos restantes: 62.40%
Porcentaje de átomos restantes: 61.60%
Porcentaje de átomos restantes: 60.80%
Porcentaje de átomos restantes: 60.00%
Porcentaje de átomos restantes: 59.20%
Porcentaje de átomos restantes: 58.40%
Porcentaje de átomos restantes: 57.60%
Porcentaje de átomos restantes: 56.80%
Porcentaje de átomos restantes: 56.00%
Porcentaje de átomos restantes: 55.20%
Porcentaje de átomos restantes: 54.40%
Porcentaje de átomos restantes: 53.60%
Porcentaje de átomos restantes: 52.80%
Porcentaje de átomos restantes: 52.00%
Porcentaje de átomos restantes: 51.20%
Porcentaje de átomos restantes: 50.40%
Porcentaje de átomos restantes: 49.60%
Porcentaje de átomos restantes: 48.80%
Porcentaje de átomos restantes: 48.00%
Porcentaje de átomos restantes: 47.20%
Porcentaje de átomos restantes: 46.40%
Porcentaje de átomos restantes: 45.60%
Porcentaje de átomos restantes: 44.80%
```

```
Porcentaje de átomos restantes: 44.00%
Porcentaje de átomos restantes: 43.20%
Porcentaje de átomos restantes: 42.40%
Porcentaje de átomos restantes: 41.60%
Porcentaje de átomos restantes: 40.80%
Porcentaje de átomos restantes: 40.00%
Porcentaje de átomos restantes: 39.20%
Porcentaje de átomos restantes: 38.40%
Porcentaje de átomos restantes: 37.60%
Porcentaje de átomos restantes: 36.80%
Porcentaje de átomos restantes: 36.00%
Porcentaje de átomos restantes: 35.20%
Porcentaje de átomos restantes: 34.40%
Porcentaje de átomos restantes: 33.60%
Porcentaje de átomos restantes: 32.80%
Porcentaje de átomos restantes: 32.00%
Porcentaje de átomos restantes: 31.20%
Porcentaje de átomos restantes: 30.40%
Porcentaje de átomos restantes: 29.60%
Porcentaje de átomos restantes: 28.80%
Porcentaje de átomos restantes: 28.00%
Porcentaje de átomos restantes: 27.20%
Porcentaje de átomos restantes: 26.40%
Porcentaje de átomos restantes: 25.60%
Porcentaje de átomos restantes: 24.80%
Porcentaje de átomos restantes: 24.00%
Porcentaje de átomos restantes: 23.20%
Porcentaje de átomos restantes: 22.40%
Porcentaje de átomos restantes: 21.60%
Porcentaje de átomos restantes: 20.80%
Porcentaje de átomos restantes: 20.00%
Porcentaje de átomos restantes: 19.20%
Porcentaje de átomos restantes: 18.40%
Porcentaje de átomos restantes: 17.60%
Porcentaje de átomos restantes: 16.80%
Porcentaje de átomos restantes: 16.00%
Porcentaje de átomos restantes: 15.20%
Porcentaje de átomos restantes: 14.40%
Porcentaje de átomos restantes: 13.60%
Porcentaje de átomos restantes: 12.80%
Porcentaje de átomos restantes: 12.00%
Porcentaje de átomos restantes: 11.20%
Porcentaje de átomos restantes: 10.40%
Porcentaje de átomos restantes: 9.60%
Porcentaje de átomos restantes: 8.80%
Porcentaje de átomos restantes: 8.00%
Porcentaje de átomos restantes: 7.20%
Porcentaje de átomos restantes: 6.40%
```

```
Porcentaje de átomos restantes: 5.60%
Porcentaje de átomos restantes: 4.80%
Porcentaje de átomos restantes: 4.00%
Porcentaje de átomos restantes: 3.20%
Porcentaje de átomos restantes: 2.40%
Porcentaje de átomos restantes: 1.60%
Porcentaje de átomos restantes: 0.80%
Porcentaje de átomos restantes: 0.00%
```

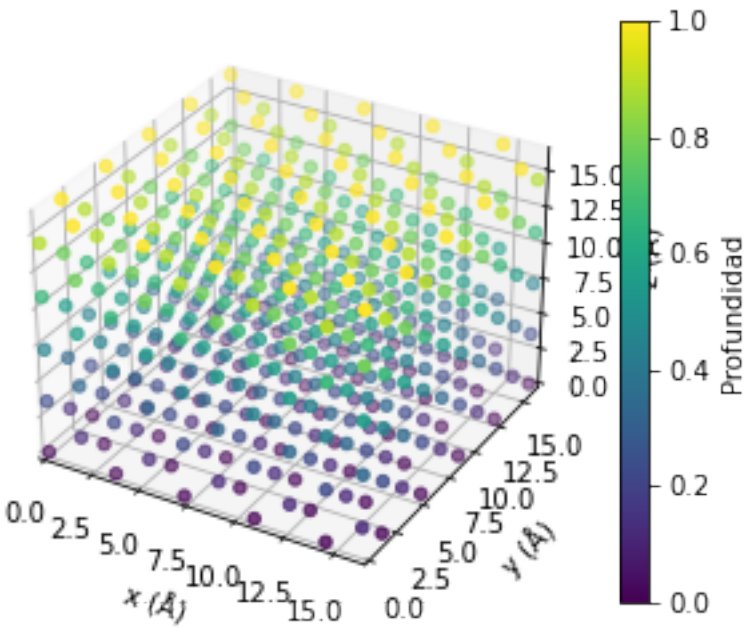
RED CÚBICA CON 5 CELDAS UNIDAD POR DIRECCIÓN.SUPERFICIES LIBRES.




```
La energía total del sistema por átomo es -1.0714817373845016 eV.
Se eligió como condiciones de contorno: superficies libres
```

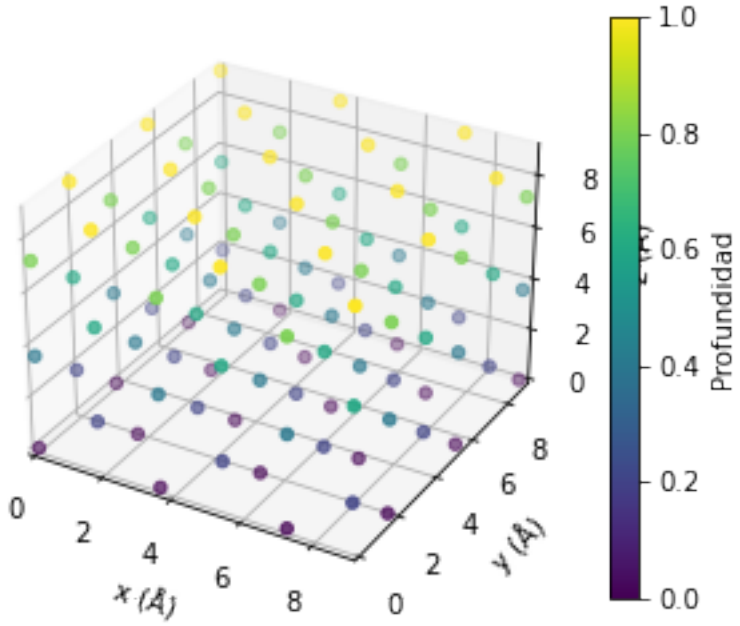
Para este primer caso se ha introducido mal las condiciones de contorno para que sea posible observar los mensajes de interacción que se han añadido para la interacción con el usuario por la terminal en caso de error.

RED CÚBICA CON 5 CELDAS UNIDAD POR DIRECCIÓN. PERIÓDICAS.




```
La energía total del sistema por átomo es -1.384972370225404 eV.
Se eligió como condiciones de contorno: periodicas
```

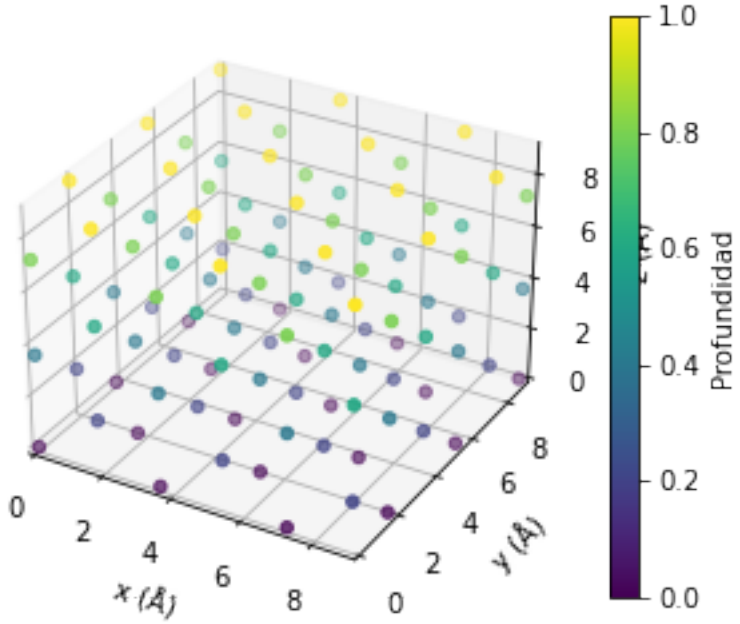
RED CÚBICA CON 3 CELDAS UNIDAD POR DIRECCIÓN. PERIÓDICAS.




```
La energía total del sistema por átomo es -1.3538647448288885 eV.
```

```
Se eligió como condiciones de contorno: periodicas
```

RED CÚBICA CON 3 CELDAS UNIDAD POR DIRECCIÓN. SUPERFICIES LIBRES.




```
La energía total del sistema por átomo es -0.8878242135535628 eV.
Se eligió como condiciones de contorno: superficies libres
```

Comentemos las energías que nos salen. Solo hemos realizado el caso 5x5x5 y 3x3x3 por no llenar todo de gráficas iguales, al final de cuentas solo nos iba a cambiar el número de átomos que observamos, y bueno la energía. Si se quieren comprobar otros casos bastará con correr el código e introducir los parámetros deseados.

En cuanto a las energías. Podemos observar como en función del número de átomos y el tipo de condiciones obtenemos unas u otras energías.

En cuanto al caso periódico, si vamos aumentado el número de celdas por dirección lo que observamos es que nos topamos con un techo de energía por átomo, es decir, un valor que parece el límite. Esto se debe al radio de corte que estamos considerando, si modificamos el radio de corte, en principio, la energía debería de tener otro techo, por decirlo de alguna manera. El radio de corte lo que nos está haciendo, en esencia, es solo considerar la interacción con aquellos vecinos que se encuentren por debajo del radio, si te encuentras fuera de este pues tu interacción con él pasa a ser nula, a más radio más vecinos consideraremos. Claro está, esto también dependerá de lo grande que es la caja.

En cuanto a las energías en condiciones de superficies libres podemos observar que es menor, y si empezáramos a aumentar el número de celdas por dirección, sin modificar el radio de corte, observaríamos que la energía se va a ir pareciendo a la de condiciones periódicas. Esto se debe al criterio que se sigue para cada una de ellas.

En superficies libres los átomos de los 'bordes' tienen menos interacciones que los del interior,

básicamente por que tienen menos vecinos. Sin embargo, en el caso periódico esto es diferente, pues si observamos el trozo de código podemos ver el criterio que se sigue. Si la distancia entre los átomos es mayor que la mitad de la longitud total de la caja se realiza una imagen de esta restándole, a la distancia real, la longitud de la caja, colocando así, de forma virtual, el átomo al lado, como si se tratara de una continuación de la misma, o en otras palabras, como si este se encontrará fuera de ésta. Esto hace, que tengamos más contribuciones por átomo y pon tanto, que la energía sea mayor, en valor absoluto.

A medida, que aumentemos el tamaño de la caja, dejando el radio de corte constante, lo que ocurrirá es que cada vez menos par de átomos, que respeten el radio de corte, cumplirán que su distancia el mayor que la mitad de la caja y por tanto, el trabajar en condiciones periódicas o superficies libres se volverá lo mismo.

Claro está, que aumentando el radio de corte, a la vez que el tamaño de la caja, esto no se observará con tanta facilidad.

EXTRA

Ahora presentaremos los principales cambios que se han realizado sobre el script de la práctica principal.

En la parte donde teniamos definidas todas las variables, ya no predefinimos el radio de corte en tres veces sigma, sino que introducimos un input para que el usuario elija.

[ ]:
```
sigma = 2.3151 # Parámetro de longitud del potencial de Lennard-Jones en Å
epsilon = 0.167 # Parámetro de energía del potencial de Lennard-Jones en eV
a = 3.603 # Parámetro de red de la red f.c.c. en Å
n = 12 # Exponente repulsivo del potencial
m = 6 # Exponente atractivo del potencial
```

También, se modifica la función energia_total. Se va guardando en cada cálculo la energía potencial de cada uno de los átomos.

[ ]:
```
def energia_total(pos, nx, ny, nz, a, periodicas, sigma, epsilon, n, m, rc):
  # Crear una variable para almacenar la energía total
  U = 0
  # Crear una lista vacía para almacenar la energía potencial de cada átomo
  U_lista = []
  # Recorrer todos los átomos
  N = len(pos) # Número total de átomos
  for i in range(N):
    # Crear una variable para almacenar la energía potencial del átomo i
    Ui = 0
    # Recorrer todos los átomos distintos de i
    for j in range(N):
      if i != j: # Evitar contar el mismo átomo
        # Calcular la energía de interacción entre los átomos i y j y sumarla a␣
↪la energía total y a la energía potencial del átomo i
        Uij = energia_interaccion(i, j, pos, nx, ny, nz, a, periodicas, sigma,␣
↪epsilon, n, m, rc)
```

```
        U += 0.5*Uij
        Ui += 0.5 * Uij # Se divide por dos para evitar contar dos veces la␣
↪misma interacción
    # Añadir la energía potencial del átomo i a la lista
    U_lista.append(Ui)
  # Devolver la energía total y la lista de energías potenciales
  return U, U_lista
```

Notar que en la función graficar_red también hemos introducido un cambio. Ahora normalizamos todos los valores teniendo en cuenta que las energías son negativas, no es igual que en el primer script donde las profundidades eran positivas. Aquí, el valor más negativo al restarle el mínimo de la lista, de energías, se hace cero mientras que el valor menos negativo será el 1. Esto es algo importante a la hora de entender nuestro gráfico coloreado 3D pues los colores asociados a valores más cercanos a cero serán los que dispongan de energías mayores, en valor absoluto, es decir, de energías más negativas mientras que los que menos interacción presentan serán aquellos que estén asociados a colores de valores más cercanos a uno. Esto se observará con facilidad pues veremos que el interior de la caja presentará colores asociados al cero mientras que los bordes a valores cercanos a 1, o incluso 1 en las esquinas (en superficies libres).

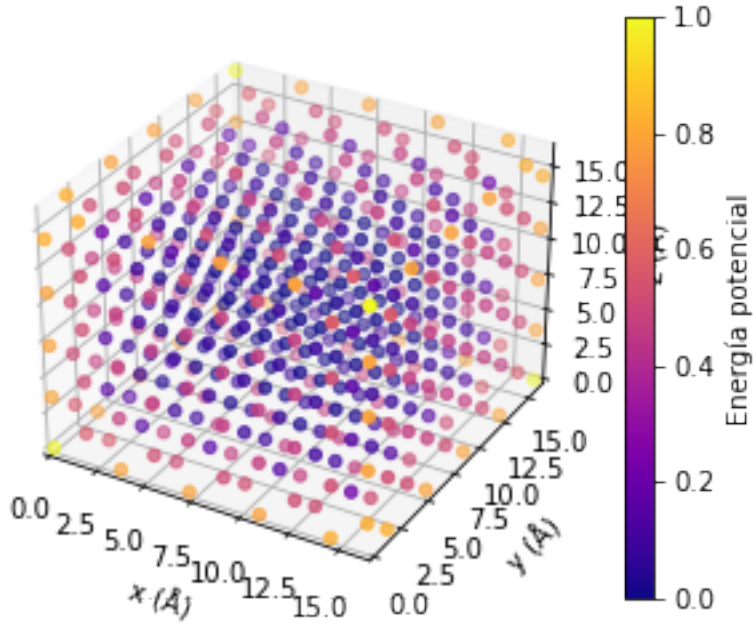
[ ]:
```
def graficar_red(pos, U_lista):
  # Crear una figura y un eje 3D
  fig = plt.figure()
  ax = fig.add_subplot(111, projection='3d')
  # Extraer las coordenadas x, y, z de la matriz de posiciones
  x = pos[:, 0]
  y = pos[:, 1]
  z = pos[:, 2]
  # Calcular el color de cada átomo según su energía potencial.
  c = (np.array(U_lista) - min(U_lista)) / (max(U_lista) - min(U_lista))

  # Representar los átomos como puntos de colores
  scatter = ax.scatter(x, y, z, c=c, cmap='plasma')

  cbar = plt.colorbar(scatter)
  cbar.set_label('Energía potencial')

  # Ajustar los límites de los ejes
  ax.set_xlim(min(x), max(x))
  ax.set_ylim(min(y), max(y))
  ax.set_zlim(min(z), max(z))
  # Etiquetar los ejes
  ax.set_xlabel('x (Å)')
  ax.set_ylabel('y (Å)')
  ax.set_zlabel('z (Å)')
  # Mostrar el gráfico
  plt.show()
```

Por último ya, en la última parte del script se añade un nuevo input, donde dejamos a elección del

usuario el radio de corte.

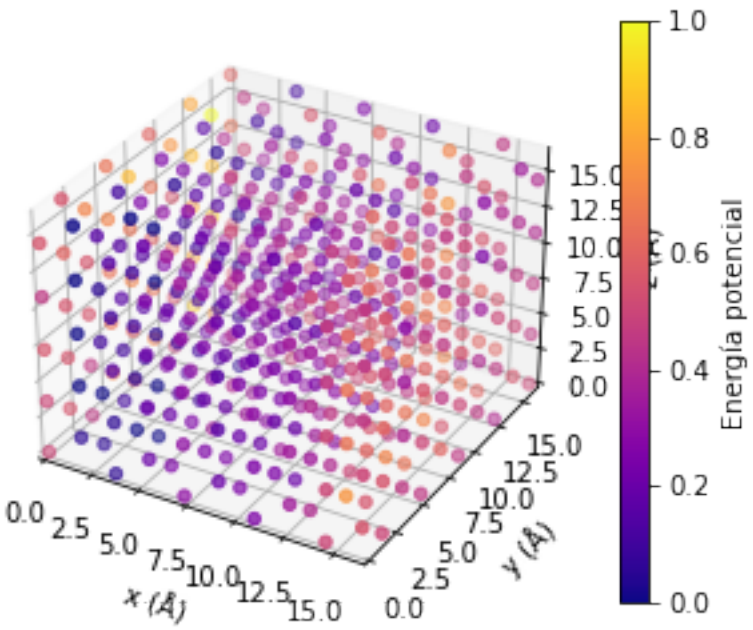
```
[ ]: rc = float(input("Introduzca el valor del radio de corte del potencial en␣
     ↪unidades de sigma: ")) * sigma
```

RED CÚBICA CON 5 CELDAS UNIDAD POR DIRECCIÓN. SUPERFICIES LIBRES.



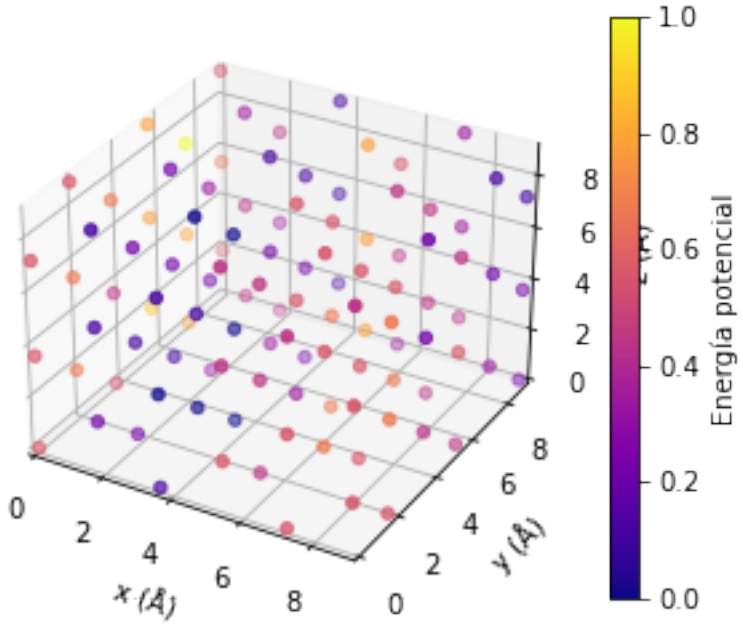

```
La energía total del sistema por átomo es -1.0714817373844845 eV.
Se eligió como condiciones de contorno: superficies libres
```

RED CÚBICA CON 5 CELDAS UNIDAD POR DIRECCIÓN. PERIÓDICAS.




```
La energía total del sistema por átomo es -1.3849723702253192 eV.
Se eligió como condiciones de contorno: periodicas
```

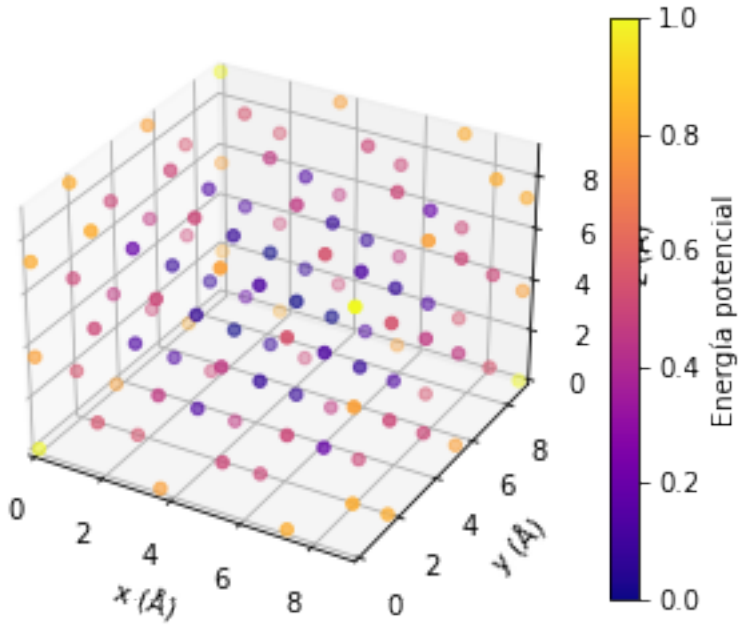
RED CÚBICA CON 3 CELDAS UNIDAD POR DIRECCIÓN. PERIÓDICAS.




```
La energía total del sistema por átomo es -1.3538647448289425 eV.
```

```
Se eligió como condiciones de contorno: periodicas
```

RED CÚBICA CON 3 CELDAS UNIDAD POR DIRECCIÓN. SUPERFICIES LIBRES.




```
La energía total del sistema por átomo es -0.8878242135535664 eV.
Se eligió como condiciones de contorno: superficies libres
```

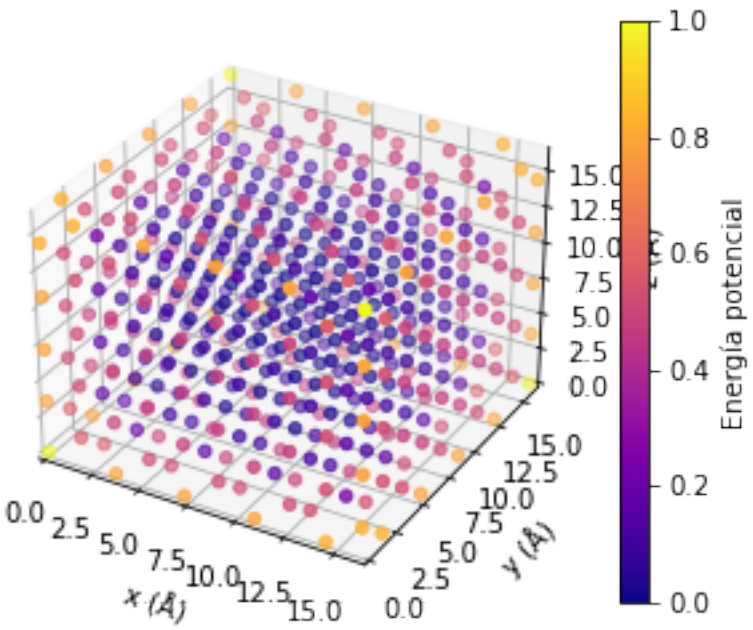
Podemos observar como las energías coinciden con los casos del script anterior, como era de esperar. Ahora podemos observar cómo cambia la energía potencial en cada uno de los átomos de la red, en función de su posición.

En las zonas más internas encontramos los átomos más cercanos al azul, pues estos son los que disponen de las energías más negativas. En cuanto a los bordes es donde encontramos los átomos con colores más cercanos a los asociados a valor unidad, es decir, los que disponen de energías menos negativas, cosa que es de esperar por disponer de menos vecinos a su alrededor.

Algo interesante a comentar es que ahora podemos observar con más claridad la influencia de dar condiciones periódicas. En los casos periódicos observamos que los átomos más externos no presentan colores tan amarillos como los que presentan, para el mismo caso, en el caso de superficies libres. Esto se debe a lo discutido más arriba. En condiciones periódicas átomos que cumplan el radio de corte y que se encuentren a una distancia mayor a la mitad de la caja serán consideradas sus imágenes al otro lado de la caja.

OBSERVEMOS ALGÚN CASO DIFERENTE A LOS ANTERIORES.

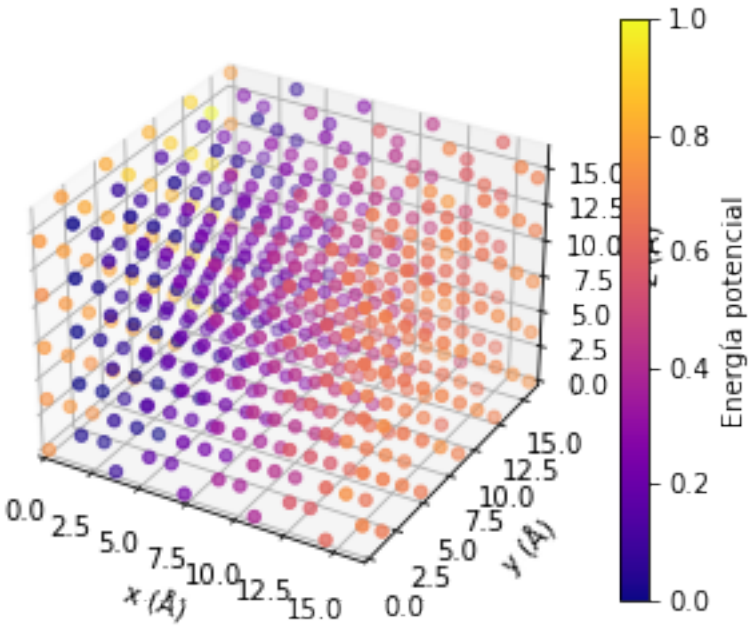
RED CÚBICA CON 5 CELDAS UNIDAD POR DIRECCIÓN. SUPERFICIES LIBRES. RADIO DE CORTE 6 VECES SIGMA




```
La energía total del sistema por átomo es -1.088299242974253 eV.
Se eligió como condiciones de contorno: superficies libres
```

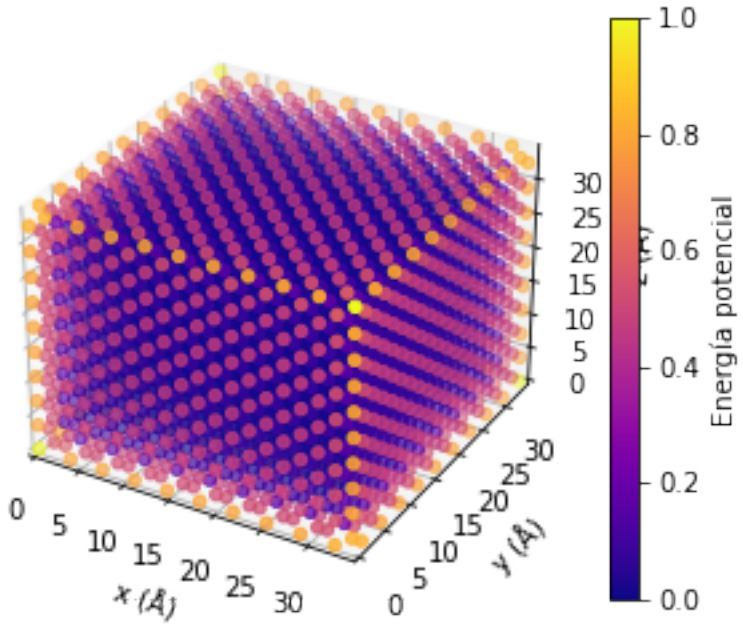
Al aumentar el radio de corte vemos que la energía por átomo aumenta. Habrán más átomos que entren dentro del radio de corte y por tanto habrán más contribuciones.

RED CÚBICA CON 5 CELDAS UNIDAD POR DIRECCIÓN. PERIÓDICAS. RADIO DE CORTE 6 VECES SIGMA



```
La energía total del sistema por átomo es -1.4177737192772117 eV.
Se eligió como condiciones de contorno: periódicas
```

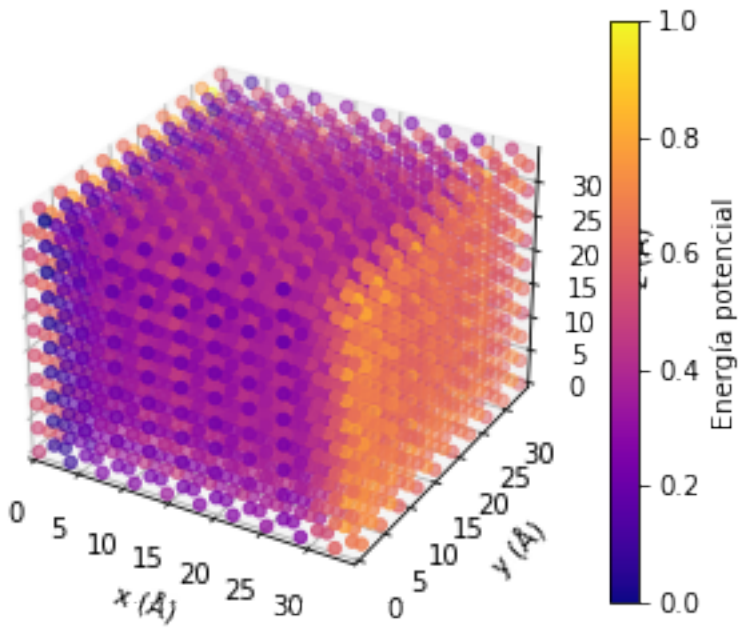
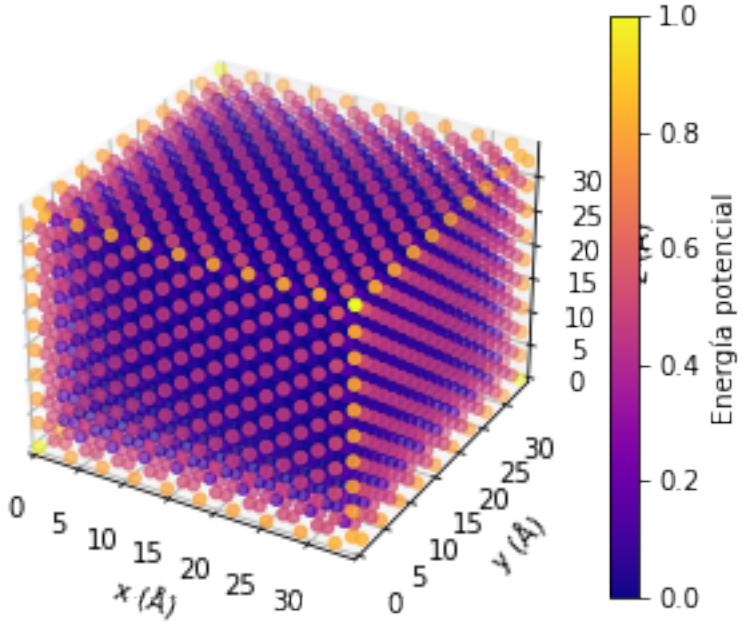
RED CÚBICA CON 10 CELDAS UNIDAD POR DIRECCIÓN. SUPERFICIES LIBRES. RADIO DE CORTE 5 VECES SIGMA



En el caso de condiciones periódicas podemos observar un error, el cual no consigo solucionar. Este error para el primer script no influye. El caso es que se observa claramente una asimetría cosa que no debería de ocurrir, al menos yo no esperaba este comportamiento. Al ser indiferente la dirección en la cual se realiza la imagen de los átomos el potencial por átomo debería ser igual en todas direcciones y no como observamos. Revisando el código creo que lo que presento en la función distancia está bien pues corrijo en cada una de las direcciones. Sin embargo, parece que solo se corrige en una de ellas, observando así el gradiente que observamos. Puede ser que el error esté en la función, modificada, energia_total. No obstante, yo no encuentro nada que me llame la atención.

En cuanto a el caso de condiciones de superficies libres vemos que sale perfecto.

RED CÚBICA CON 10 CELDAS UNIDAD POR DIRECCIÓN. PERIÓDICAS. RADIO DE CORTE 5 VECES SIGMA



RED CÚBICA CON 10 CELDAS UNIDAD POR DIRECCIÓN. SUPERFICIES LIBRES. RADIO DE CORTE 3 VECES SIGMA